

Hackceler8 2026

Input Replayer Manual

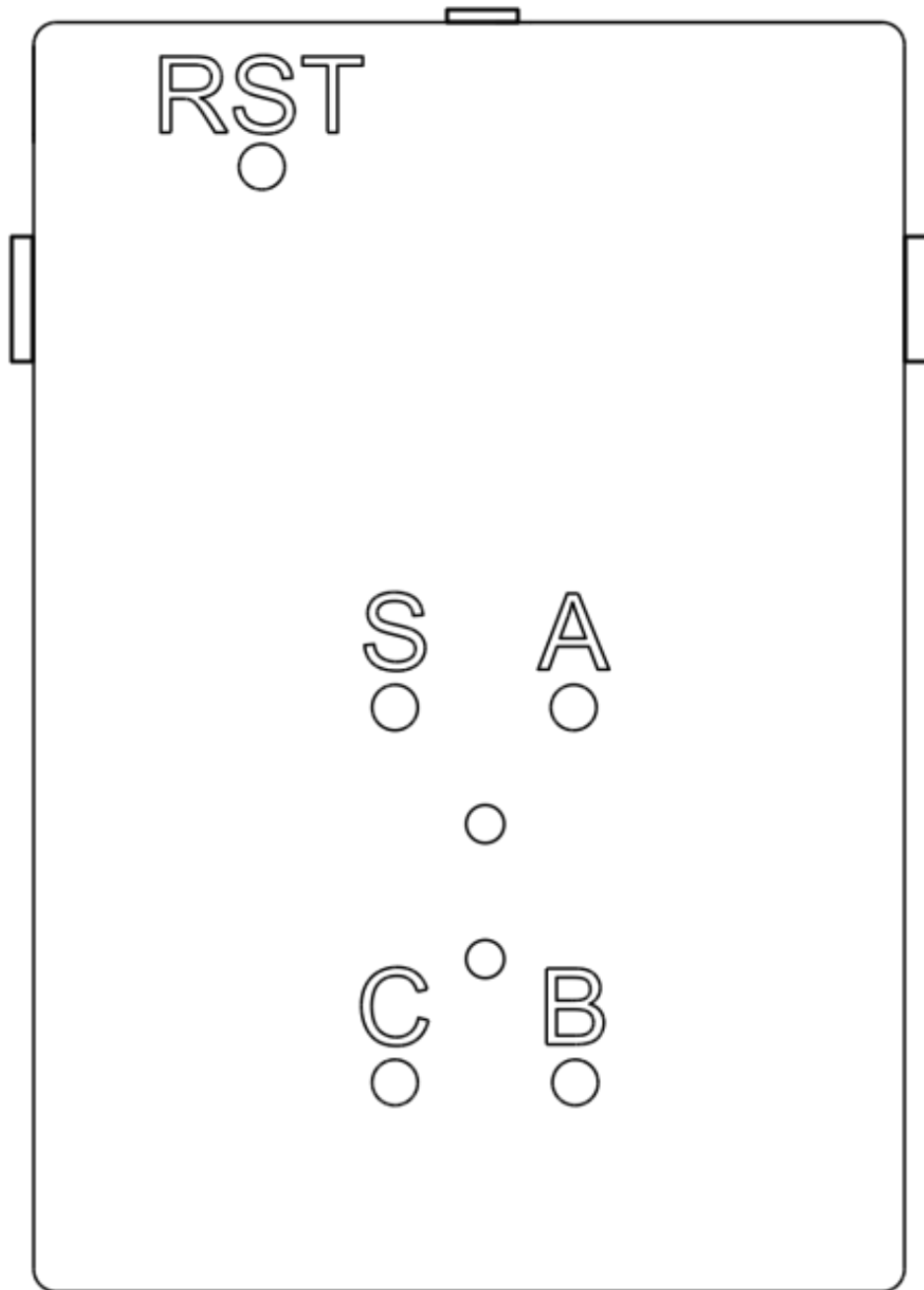


Table of contents

Introduction.....	3
Overview.....	4
Concepts.....	5
Usage.....	6
Appendix A - Replay File Format.....	7

Introduction

Welcome to Hackceler8 2026!

Just like last year, you will play this year's edition on the Mega Drive. This year there is an important difference, however: we will provide you with a way to send pre-programmed input to the console. To this end we've created a device which lets you send pre-programmed input to the console, which we call the replayer.

The replayer lets you create or record a pre-programmed sequence of controller inputs on your PC and play it back on the console with frame-perfect accuracy. It supports branching traces to select between multiple different input sequences at runtime and can sync with a console reset to let you precisely control the game from the very first frame.

This document explains how the replayer works and how you can use it. Each team will have access to a replayer during the competition, and using it effectively will be important for your team's success, so make sure to read it!

We've tried our best to make sure this document is accurate and the replayer is free from bugs, but as we all know, mistakes happen. Should you have any questions or find any bugs or problems, feel free to contact Nspace on Discord (nspace8488) or in person during the event. He will be happy to answer any questions.

Happy hacking!

The Hackceler8 2026 team.

Overview

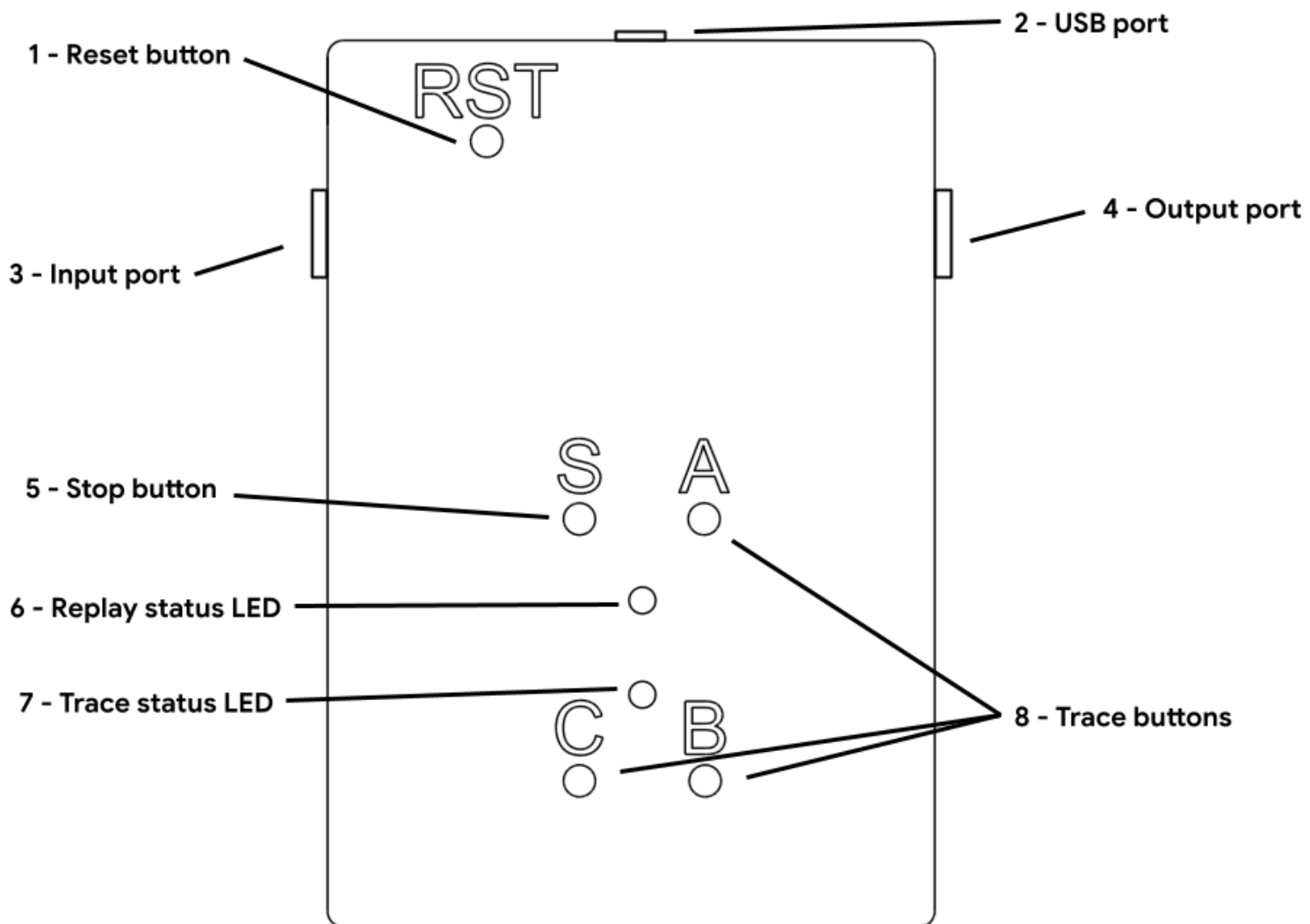


Fig. 1: overview of the replayer.

1. **Reset button** - press this to reset the replayer. You shouldn't need to use this, but it can be useful in case there is a bug.
 - **Warning:** resetting the replayer wipes the uploaded file.
2. **USB port** - connect this to your laptop.
3. **Input port** - connect a controller here.
4. **Output port** - connect this to the console's controller port.

5. **Stop button** - stops the current replay immediately.

6. **Replay status LED**

- Off when the replayer is idle.
- Fixed green when the replayer is playing back a trace.
- Blinking green when the replayer is waiting for a console reset.
- Fixed blue when the replayer is waiting for you to select a trace.
- Fixed red when the replayer has reached the end of the loaded file.

7. **Trace status LED**

- Off when there is no trace loaded
- Fixed green when a valid file is loaded.
- Briefly blinks green when the host PC writes a valid file.
- Briefly blinks red when the host PC writes an invalid file.

8. **Trace buttons** - select a trace to play.

Concepts

- A **frame** is one iteration of the game loop. Every frame the game reads the state of the controllers and updates the state of the game in response to the input it receives.
- An **input** is a combination of button presses which is sent to the console at a specific frame.
- A **trace** is a continuous sequence of inputs that the replayer sends to the console, one per frame.
- A **replay file** is a file containing a set of traces, encoded in a format which the replayer understands.

It's possible that the challenge you're trying to solve with the replayer only requires sending a certain sequence of inputs to the game, and that this sequence of inputs doesn't depend on anything that happens in the game. In this case your replay file can consist of a single trace which gets played from start to finish and nothing else.

It's also possible that the challenge requires reacting to what's happening on the screen. In order to support this case the replayer also supports multistage files, which consist of multiple traces which you can trigger manually using the replayer's button.

- A **stage** is a set of exactly 3 traces. You select which of the 3 traces to play by pressing one of the 3 trace buttons.
- A **multistage file** consists of a sequence of stages. At each stage you select one of the 3 traces using the trace buttons, then the replayer moves on to the next stage.

Here is an example trace made of two stages:

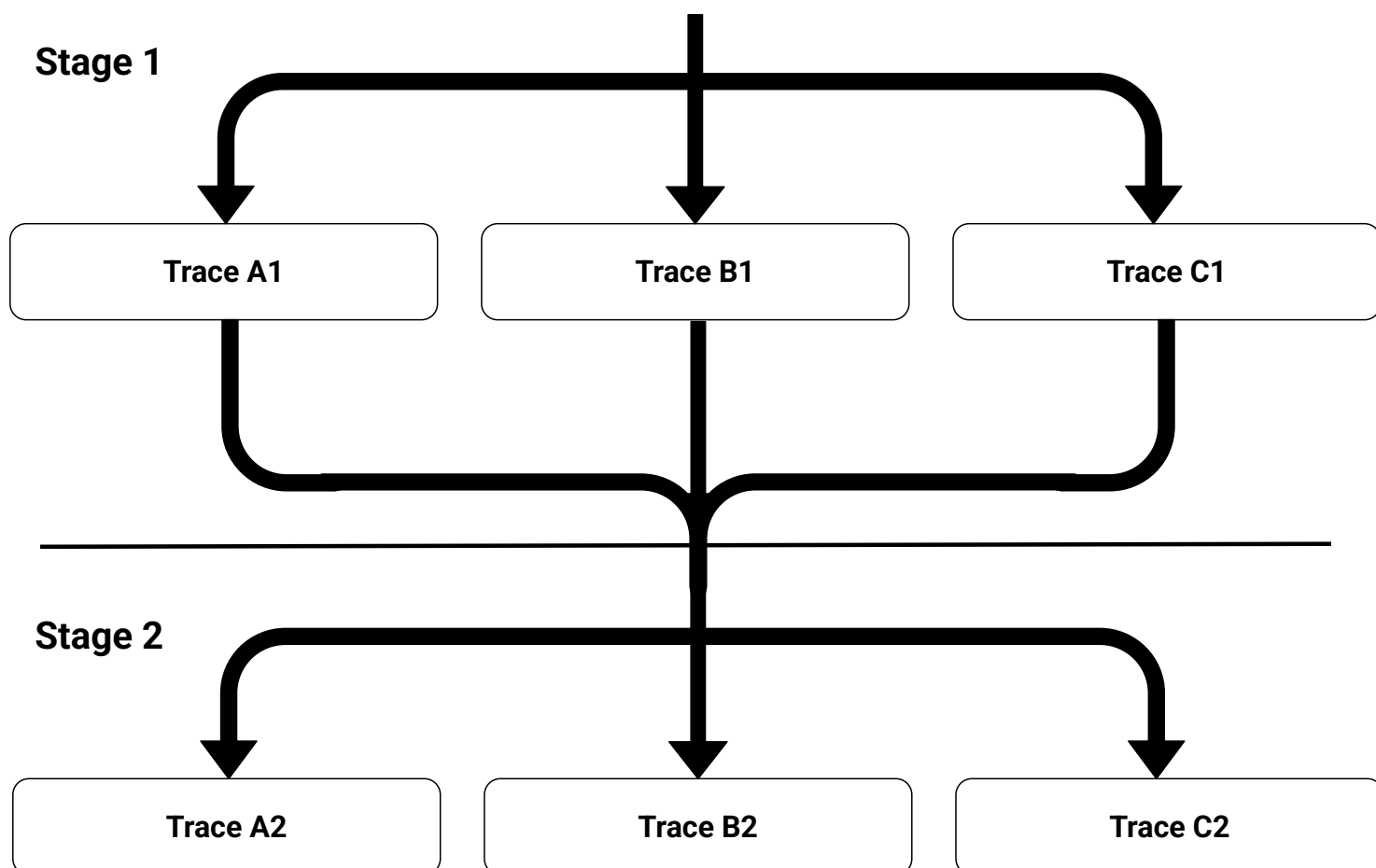


Fig. 2: an example multistage trace

If you load this file onto the replayer and press A, B, the replayer will play trace A1 followed by trace B2. Similarly, if you play C, C, the replayer will play trace C1 followed by trace C2. You can't play more than one trace from the same stage: the replayer simply ignores the unselected traces and moves on to the next stage. If you press a trace selection button while the replayer is already playing something, the newly selected trace is not played immediately, but added to a queue. Stages are always played in order (a trace from stage 1 always plays before a trace from stage 2).

Usage

Setup

1. Connect the replayer's output port to the console's first controller port (the one on the left) using the extension cable.
2. Connect a controller to the replayer's input port. The replayer (and the game) only support 3-button controllers. If a 6-button controller is plugged in, the extra 3 buttons (X, Y and Z) are ignored.
3. Connect the replayer to your laptop using a USB cable. The green power LED will turn on.
4. At startup, no file is loaded in the replayer and the replayer is idle. The trace and replay status LEDs are off. The controller is connected to the output, and you can use it to play the game on the console.
5. The replayer shows up as a FAT-formatted USB mass storage device on your PC called "REPLAYER". The virtual drive contains a README.TXT file which briefly explains how to use the replayer.

Creating a replay file

There are two ways to create a replay file: you can either use the MAME plugin to create a recording of yourself playing and convert it to a replay file, or create a trace programmatically. We provide [builder.py](#), a library which is designed to help you do the latter.

Let's create a simple file with a single trace which holds A down for 50 frames:

```
Python
from builder import File, Stage, Trace, Frame, Buttons, TraceFlags

with open('replay.bin', 'wb') as f:
    r = File([Stage(a=Trace([
        Frame(0, buttons=Buttons.A),
        Frame(50, buttons=Buttons.NONE),
    ]))])
    f.write(r.build())
```

Our example trace only contains the two frames where the state of the controllers differs from the previous frame, 0 and 50. Whenever there is a gap, the replayer keeps sending the last input it encountered in the trace until it reaches the next.

Loading and playing back a replay file

Running this script produces a replay file called `replay.bin`. Load it to the replayer by copying it to the virtual mass storage drive. The replayer's trace status LED should briefly blink green, and then stay green to indicate that you uploaded a valid file. The replay status LED should turn blue to indicate that the replayer is idle and is waiting for you to start a trace.

If you now press the A button, the replayer will play the trace from start to finish (holding A down for about one second). While the replayer is playing back the trace, the controller is disconnected from the console, so pressing buttons on the controller has no effect. The replay status LED will also become green. When the trace finishes playing, the controller will become active again.

In this case our file only has one stage, so after the trace finishes playing the status LED will turn red, indicating that the replayer has reached the end of the trace and there are no more traces to play in the current file. You can rewind to the beginning by pressing the stop button, which rewinds the current file to the beginning. If you do that the replay status LED will become blue again. Pressing the stop buttons will also immediately stop any ongoing replay and reactivate the controllers.

Note that every stage always has 3 traces. In this case we've left trace B and C empty, so if you press the B or C buttons the replayer will not send any inputs and immediately advance to the next stage. In this case there is no next stage, so the replay status LED simply turns red.

If you want to upload a different replay file, simply copy it to the virtual USB drive. Note that the replayer doesn't care about the name or extension of the file, or whether there are multiple files in the virtual drive: it will simply look at the latest file you wrote.

There can only be one file loaded on the replayer at any given time. If you upload a new valid file, the replayer will replace the old one, stopping a replay if it's ongoing. If you delete the uploaded file from the virtual USB drive, the replayer will be left with no active file and will stop replaying. In this case the status LEDs will turn off. If you upload an invalid replay file the replayer will ignore and keep the one that is currently loaded. The trace status LED will briefly flash red to indicate that the file you just uploaded was invalid.

NOTE: the files uploaded to the replayer are only saved in its RAM. Resetting or unplugging the replayer will **erase** the active file and all the contents of the virtual disk.

Converting an emulator recording

You can convert a recording made with the emulator plugin (.mlen file) to a file with a single trace that plays back that recording.

```
Python
from builder import File, Stage, Replay

with open('replay.bin', 'wb') as f:
    r = File([Stage(a=Replay('./output.mlen'))])
    f.write(r.build())
```

Starting from a reset

You can configure the replayer to wait for the console to reset before it starts to play back a trace. This is useful when you want complete control over the game state from the very first frame. You can do this by setting a flag on that trace:

```
Python
from builder import File, Stage, Trace, Frame, Buttons, TraceFlags

with open('replay.bin', 'wb') as f:
    r = File([
        Stage(
            a=Trace([
                Frame(0, buttons=Buttons.A),
                Frame(50, buttons=Buttons.NONE),
            ], flags=TraceFlags.WAIT_FOR_RESET)
        ])
    f.write(r.build())
```

When you upload this file and press trace button A the replayer won't immediately do anything and the controllers will remain enabled. The replay status LED will flash in green to indicate that the replayer is waiting for the console to reset. Press the reset button on the console and the replayer will start replaying the trace like before.

NOTE: some game features like the pause menu which don't poll the controllers for several frames can trigger the reset detection and make the replayer think the

console has reset. Make sure that you don't use those while the replayer is waiting for a reset.

Multistage files

The replayer supports files with up to 255 stages, each with 3 traces. Here is an example of how to create a multistage file:

Python

```
from builder import File, Stage, Trace, Frame, Buttons, TraceFlags

with open('replay.bin', 'wb') as f:
    r = File([
        Stage(
            a=Trace([
                Frame(0, buttons=Buttons.A),
                Frame(50, buttons=Buttons.NONE),
            ], flags=TraceFlags.REPLAY_IMMEDIATELY),
            b=Trace([
                Frame(0, buttons=Buttons.B),
                Frame(50, buttons=Buttons.NONE),
            ], flags=TraceFlags.REPLAY_IMMEDIATELY),
        ),
        Stage(
            a=Trace([
                Frame(0, buttons=Buttons.A),
                Frame(50, buttons=Buttons.NONE),
            ], flags=TraceFlags.REPLAY_IMMEDIATELY),
            b=Trace([
                Frame(0, buttons=Buttons.B),
                Frame(50, buttons=Buttons.NONE),
            ], flags=TraceFlags.REPLAY_IMMEDIATELY),
        ),
    ])
    f.write(r.build())
```

The replayer internally maintains a trace queue. When you press a button, the corresponding trace from the next unqueued stage is added to the queue, and the replayer plays the selected traces one after the other in stage order. Here the next unqueued stage refers to the first stage that you haven't yet selected a trace from.

When the replayer gets to the end of a trace and there are still traces in the queue, the replayer immediately dequeues that trace and plays it (unless that trace waits for a reset, in which case it starts waiting). If the queue is empty when the replayer finishes playing a trace, it re-enables the controllers and goes back to idle, waiting for

you to enqueue more traces. When the replayer is idle, the replay status LED indicates whether there are still traces to enqueue in the current file (blue) or whether you've reached the end of the file (red).

If you press the trace buttons while a trace is already playing, the replayer will add the trace to the trace queue, instead of playing it right away. Enqueueing empty traces is supported but they don't do anything except possibly waiting for a reset. In our example if you pressed trace button A immediately followed by trace button B, the replayer would hold A down for 50 frames and then immediately after that hold B down for 50 frames.

Appendix A - Replay File Format

This chapter describes the binary format of a replay file. `builder.py` in the game's `plugins` folder contains code which you can use to programmatically generate a correctly-formatted replay file, but we also provide a description here for your convenience.

A replay file consists of one `replay_file` struct. All integers in the file use the little-endian byte order and all structs are packed, with no padding between members.

The maximum size of a replay file is 131072 bytes. The maximum number of stages is 255. The maximum total number of inputs in a replay file, across all stages and traces is 16000.

```
struct replay_file {
    uint8_t num_stages;

    struct replay_stage stages[num_stages];
};

struct replay_stage {
    struct replay_trace traces[3];
};

struct replay_trace {
    // Bitmask: 1 = wait for reset, others not used.
    uint8_t flags;

    // How many inputs (can be 0 for an empty trace).
    uint32_t num_inputs;

    struct replay_input inputs[num_inputs];
};
```

```

struct replay_input {
    // Count from the start of the trace, zero-based.
    uint32_t frame_number;

    // Only the first element is used.
    struct buttons controller_states[4];
};

// uint8_t bitfield
struct buttons {
    // LSB
    uint8_t b : 1;
    uint8_t c : 1;
    uint8_t down : 1;
    uint8_t left : 1;
    uint8_t right : 1;
    uint8_t up : 1;
    uint8_t a : 1;

    // MSB
    uint8_t start : 1;
};

```